

Building and Securing Serverless Applications Using AWS Lambda and Amazon API Gateway

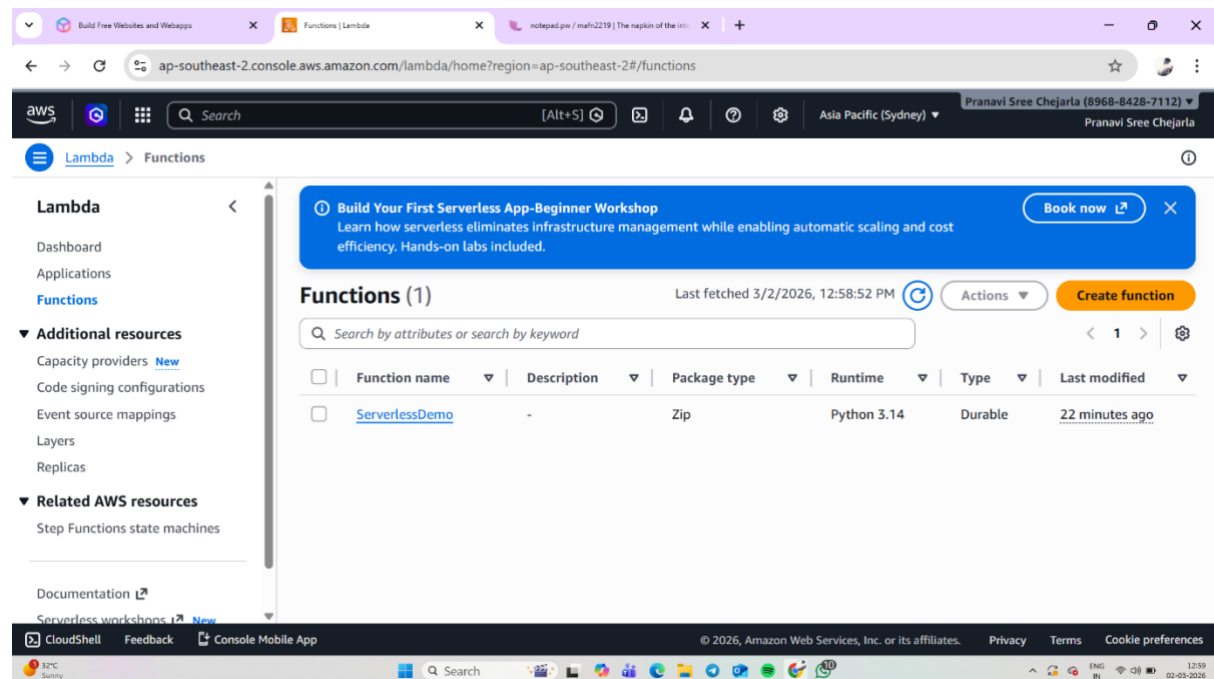
PART 1: Create Lambda Function

Step 1 – Login to AWS Console

Login to the **AWS Management Console** and search for **Lambda**.

Step 2 – Create Function

Click **Create Function** in the Lambda dashboard.



Step 3 – Choose Function Type

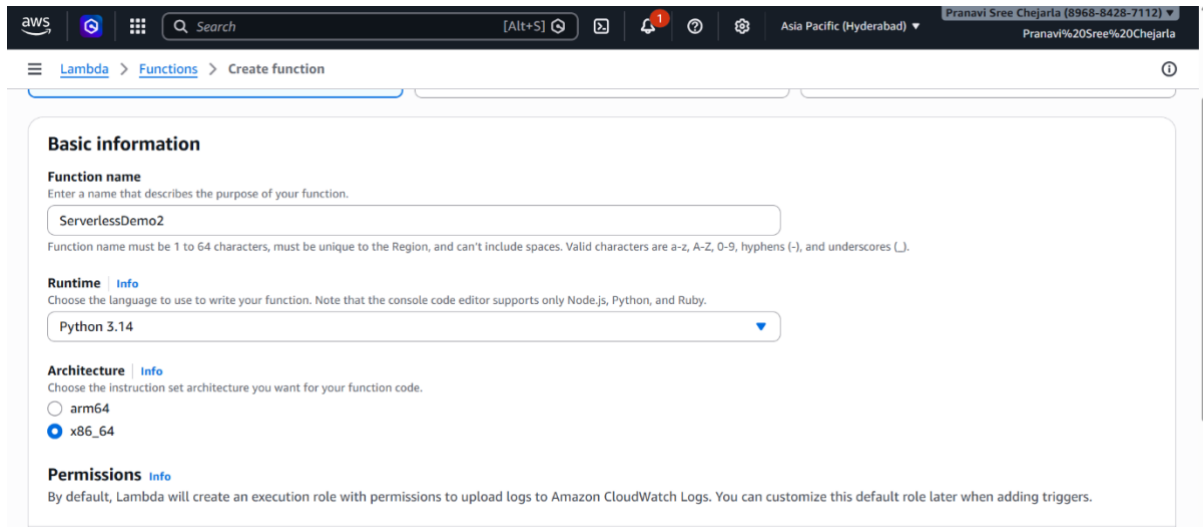
Select **Author from scratch**.

Step 4 – Configure Function

Fill the following details:

- **Function Name:** ServerlessDemo
- **Runtime:** Python 3.x
- **Architecture:** x86_64
- **Permissions:** Create new role with basic Lambda permissions

Then click **Create Function**.



Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

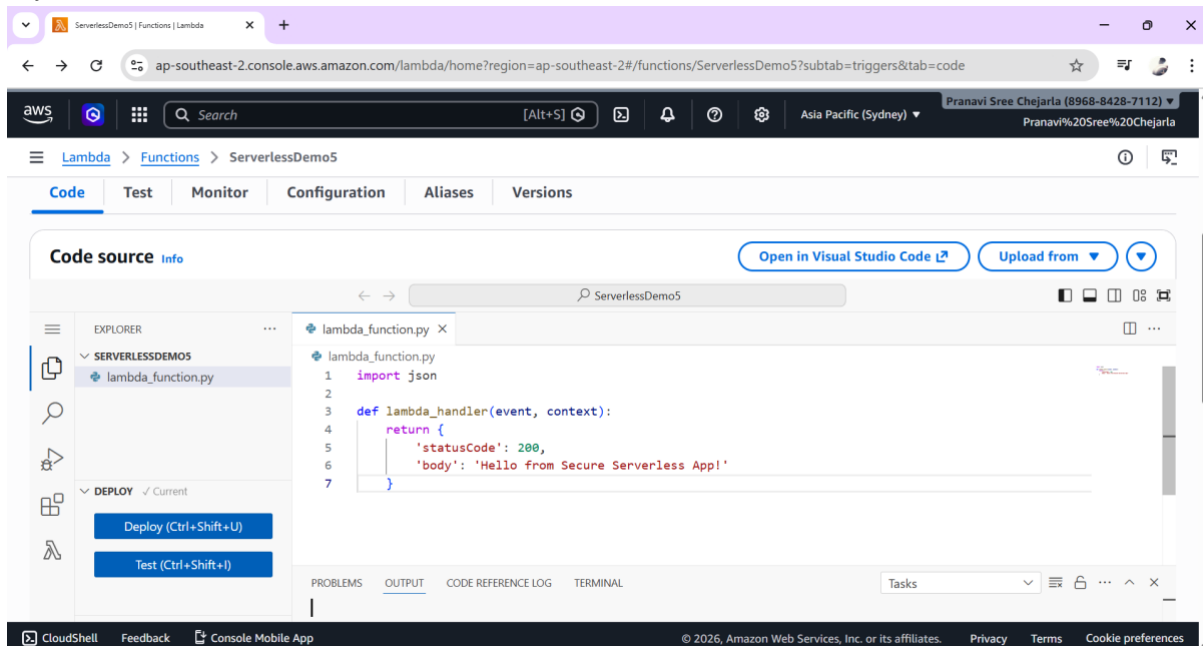
Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☐ arm64
☒ x86_64

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

Step 5 – Write Lambda Code

Replace the default code with:

```
def lambda_handler(event, context):  
  
    return {  
  
        'statusCode': 200,  
  
        'body': 'Hello from Secure Serverless App!'  
  
    }
```



Step 6 – Deploy Function

Click **Deploy** to save and activate the Lambda function.

PART 2: Test Lambda Function

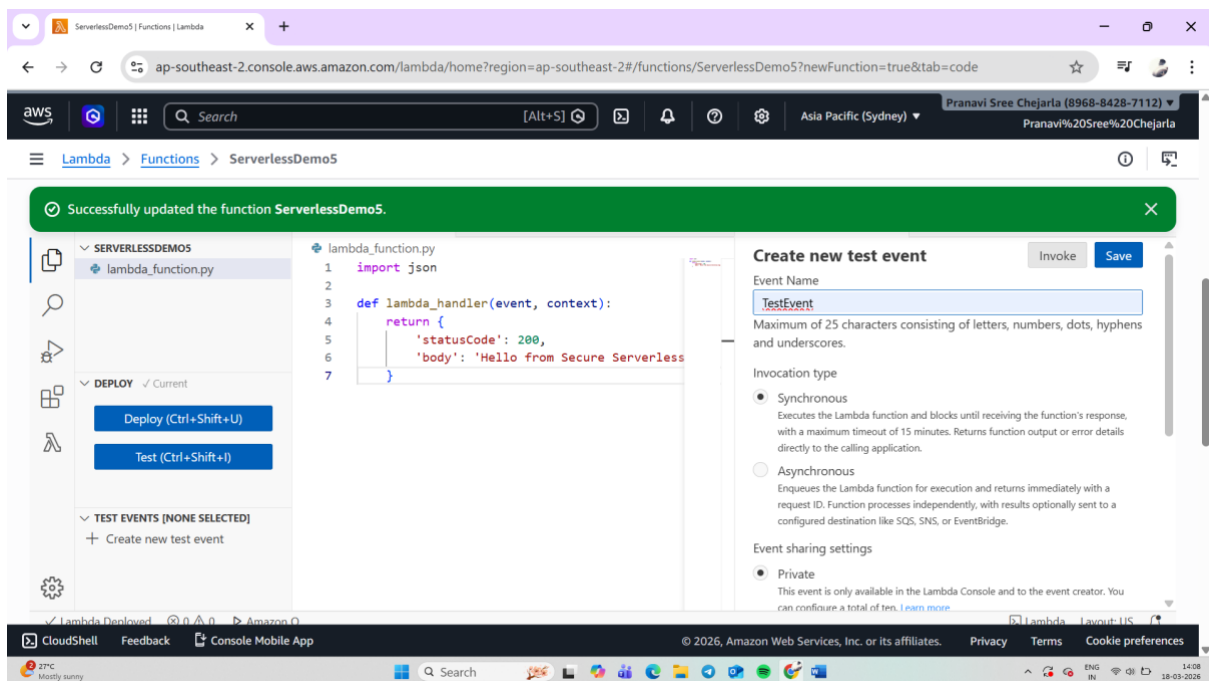
Step 7 – Click Test

Click the **Test** button in the Lambda console.

Step 8 – Create Test Event

Create a new test event named:

TestEvent



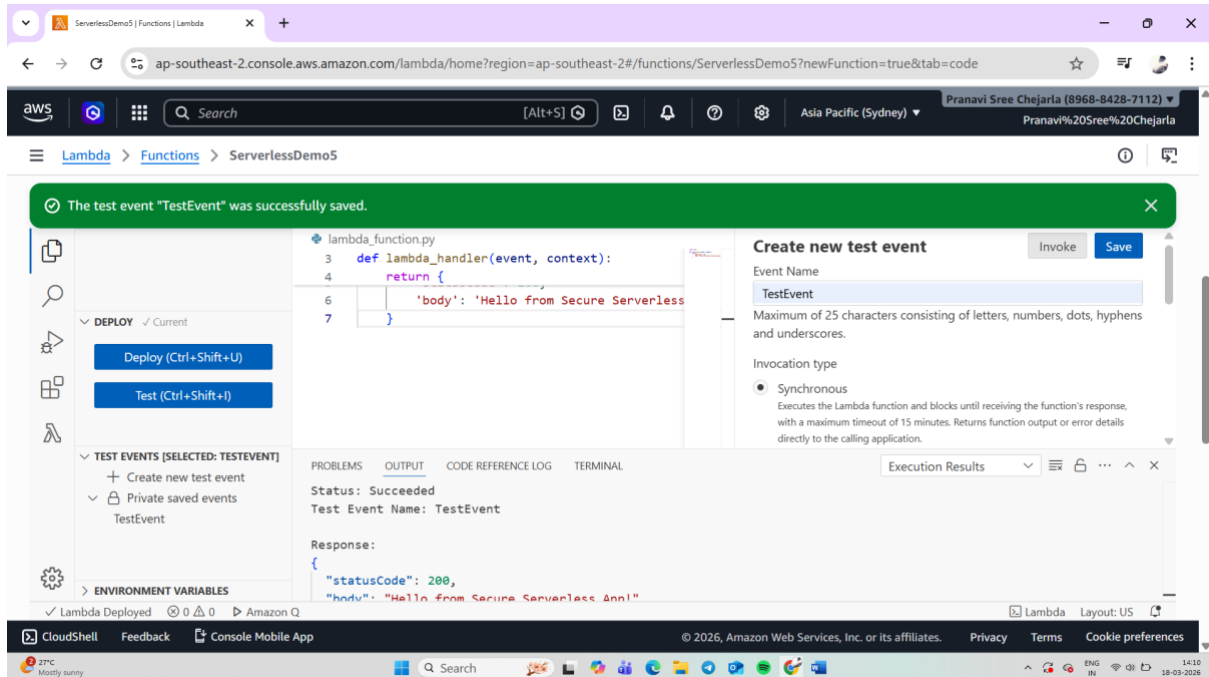
Step 9 – Execute Test

Click **Test** again to run the Lambda function.

Step 10 – Verify Output

Expected result:

Status Code: 200



Hello from Secure Serverless App!

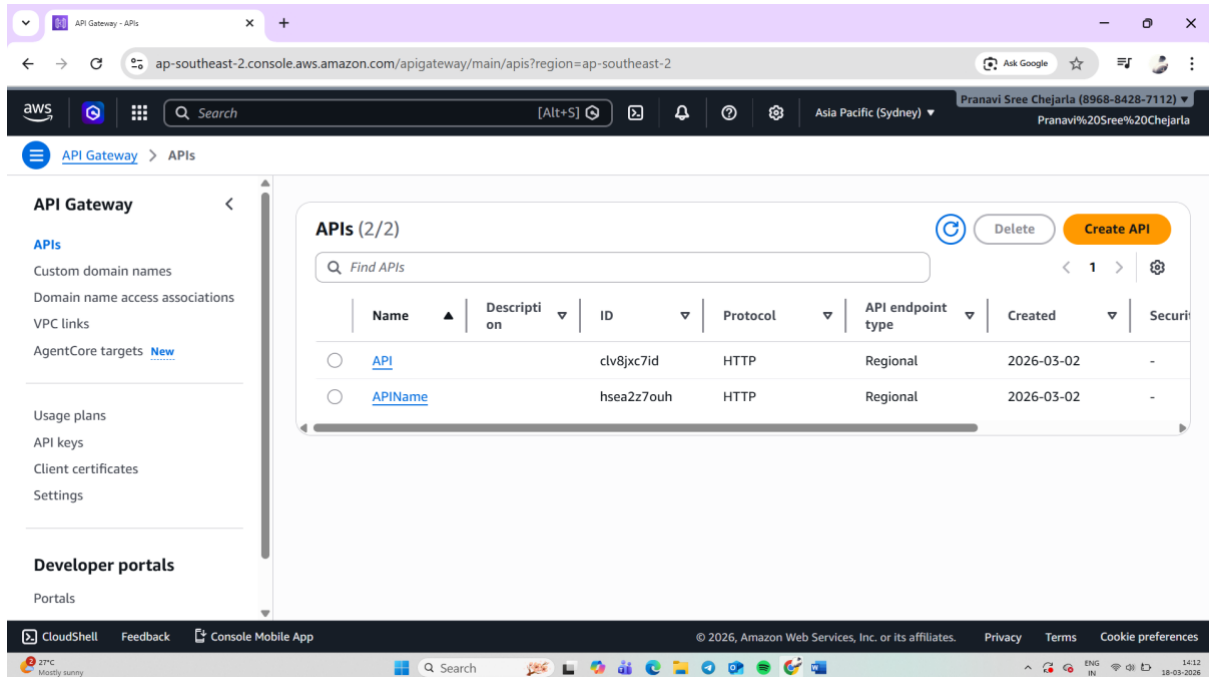
PART 3: Create API Gateway

Step 11 – Open API Gateway

Go to **API Gateway** service from AWS console.

Step 12 – Create API

Click **Create API**.



The screenshot shows the AWS API Gateway console in the 'APIs' section. The left sidebar contains navigation links for 'API Gateway' (with a sub-link for 'APIs'), 'Custom domain names', 'Domain name access associations', 'VPC links', 'AgentCore targets', 'Usage plans', 'API keys', 'Client certificates', 'Settings', and 'Developer portals'. The main content area displays a table of APIs with the following columns: Name, Description, ID, Protocol, API endpoint type, Created, and Security. Two APIs are listed: 'API' with ID 'clv8jxc7id' and 'APIName' with ID 'hsea2z7ouh'. Both are HTTP protocols with a 'Regional' endpoint type, created on '2026-03-02'. At the top right of the table, there are buttons for 'Delete' and 'Create API'. The browser address bar shows the URL 'ap-southeast-2.console.aws.amazon.com/apigateway/main/apis?region=ap-southeast-2'.

Name	Description	ID	Protocol	API endpoint type	Created	Security
API		clv8jxc7id	HTTP	Regional	2026-03-02	-
APIName		hsea2z7ouh	HTTP	Regional	2026-03-02	-

Step 13 – Choose API Type

Select **HTTP API**.

Step 14 – Build API

Click **Build**.

Step 15 – Add Integration

Select **Lambda Integration**.

Step 16 – Choose Lambda Function

Select:

ServerlessDemo

The screenshot shows the AWS API Gateway console in the 'Create HTTP API' wizard, Step 1: API details. The breadcrumb navigation is 'API Gateway > APIs > Create API > Create HTTP API'. The left sidebar shows the progress: 'Configure routes' (selected), 'Step 3 - optional', 'Define stages', 'Step 4', and 'Review and create'. The main content area is titled 'API details' and includes the following sections:

- API name:** A text input field containing 'API1'.
- IP address type:** A section with a description and two radio button options: 'IPv4' (selected) and 'Dualstack'.
- Integrations (1):** A section with a description and a table of integrations. The table has columns for 'AWS Region', 'Lambda function', and 'Version'. The first integration row shows 'ap-sou...' for the region, 'arn:aws:lambda:ap-southeast-2:8968842871' for the function, and '2.0' for the version. There is a 'Remove' button next to the integration.

The bottom of the console shows the AWS status bar with the region 'Asia Pacific (Sydney)', the user 'Pranavi Sree Chejarla', and the date '18-03-2026'.

Step 17 – Create Route

Configure the route:

- **Method:** GET
- **Path:** /hello

The screenshot shows the AWS API Gateway console in the 'Create HTTP API' wizard, Step 2: Configure routes. The breadcrumb navigation is 'API Gateway > APIs > Create API > Create HTTP API'. The left sidebar shows the progress: 'Step 1', 'Configure API', 'Step 2 - optional' (selected), 'Configure routes', 'Step 3 - optional', 'Define stages', 'Step 4', and 'Review and create'. The main content area is titled 'Configure routes - optional' and includes the following sections:

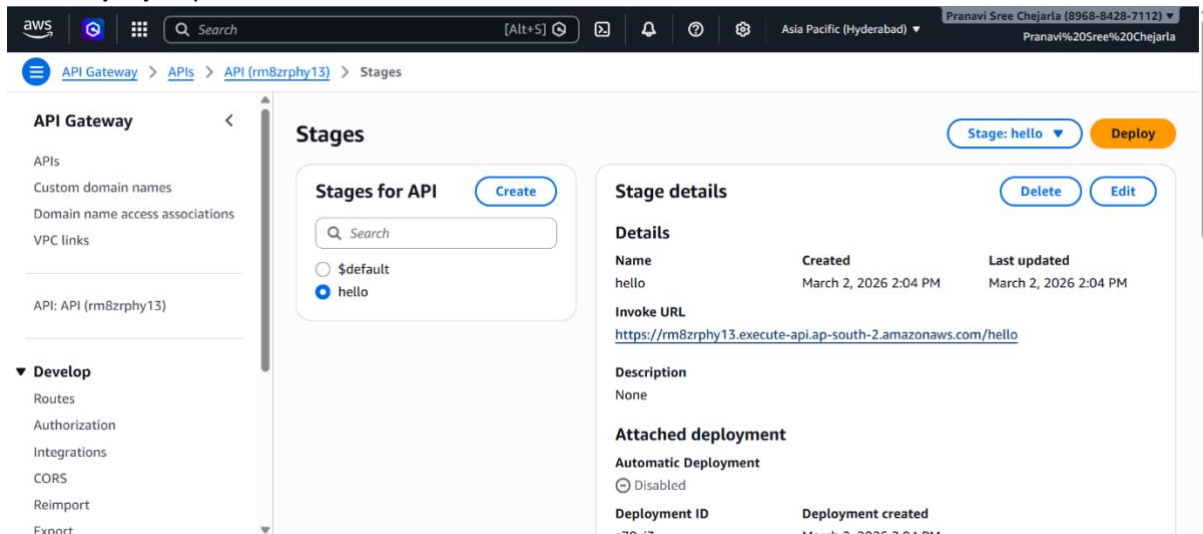
- Configure routes:** A section with a description and a table of routes. The table has columns for 'Method', 'Resource path', and 'Integration target'. The first route row shows 'GET' for the method, '/hello' for the resource path, and 'ServerlessDemo5' for the integration target. There is a 'Remove' button next to the route.

At the bottom of the main content area, there is an 'Add route' button. Below the main content area, there are four buttons: 'Cancel', 'Review and create', 'Previous', and 'Next'.

The bottom of the console shows the AWS status bar with the region 'Asia Pacific (Sydney)', the user 'Pranavi Sree Chejarla', and the date '18-03-2026'.

Step 18 – Deploy API

Click **Deploy** to publish the API

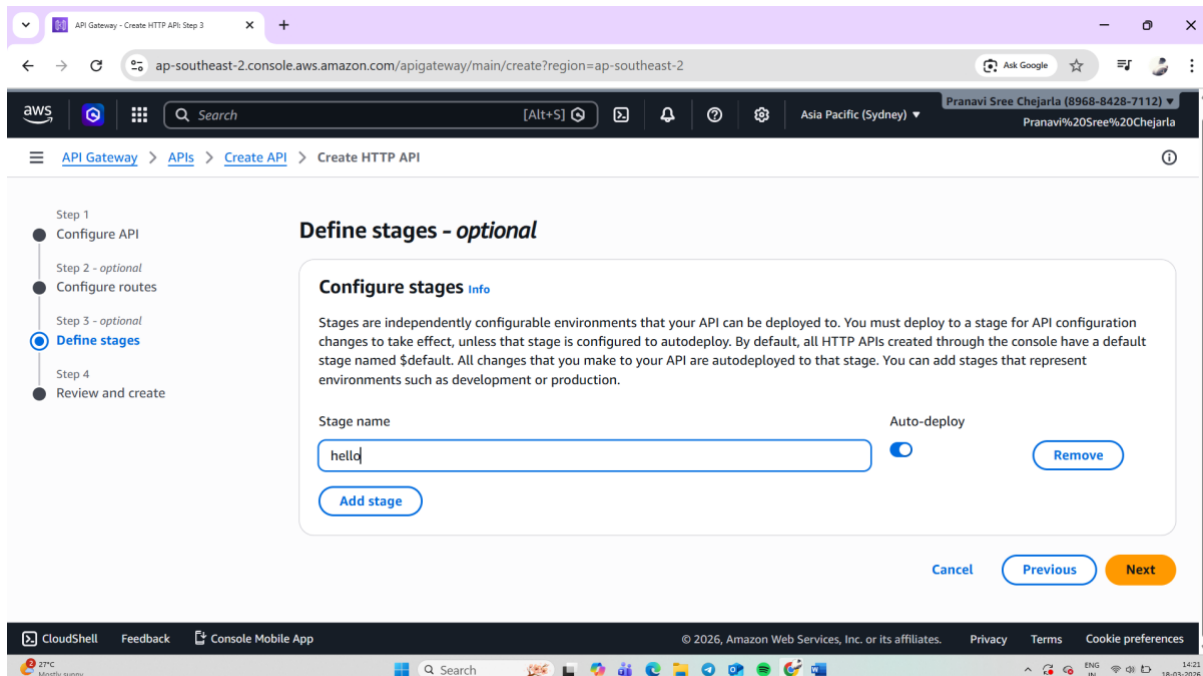


PART 4: Configure Stage

A **stage** is created to deploy and access the API.

Example stage name:

hello



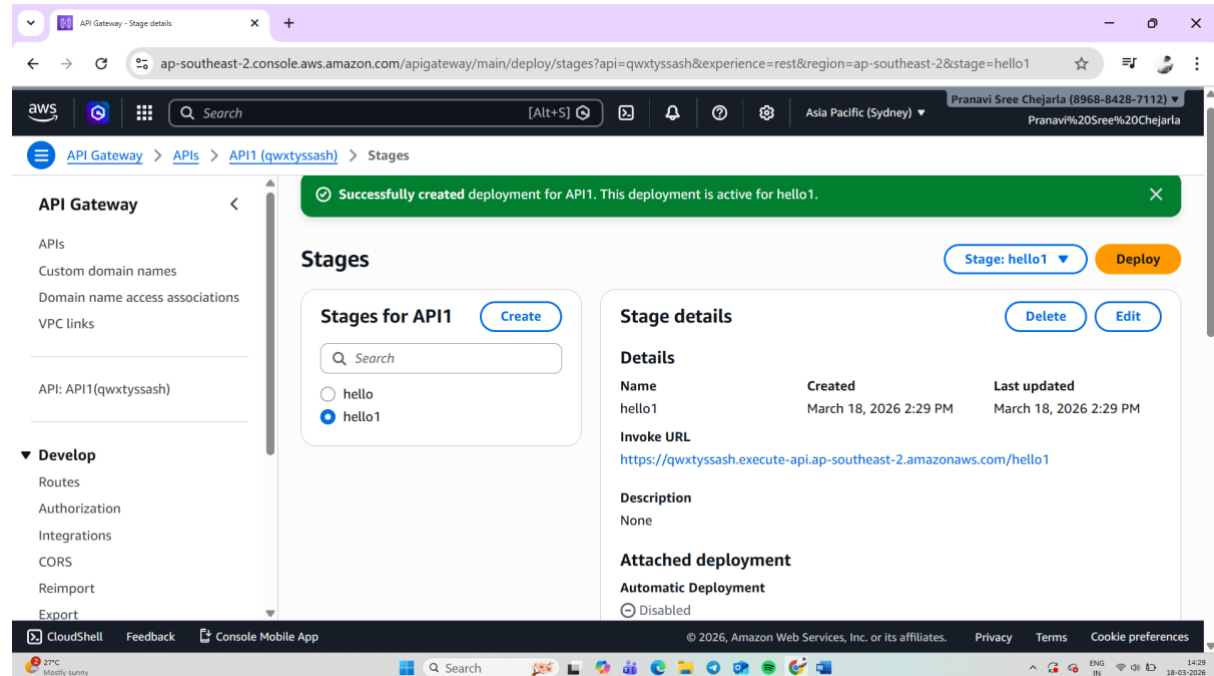
You can also edit stage configuration such as deployment settings.

PART 5: Test API Endpoint

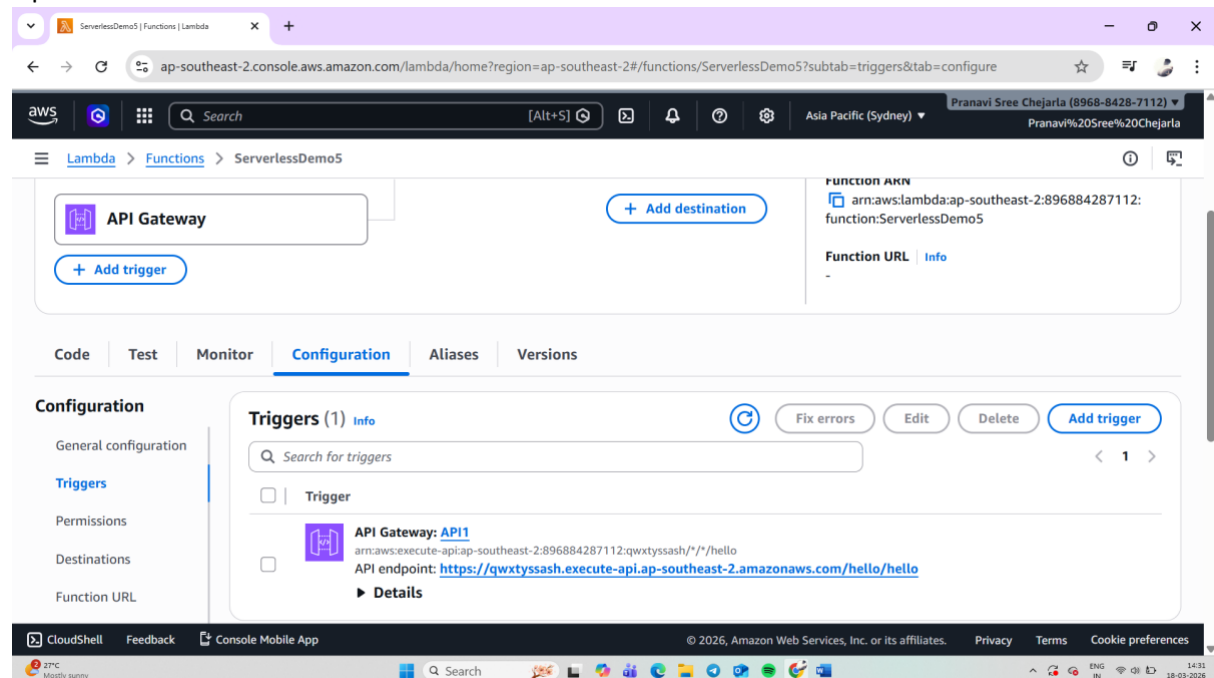
After deployment, API Gateway provides an **Invoke URL**.

Example:

<https://your-api-id.execute-api.region.amazonaws.com/hello>



Open this URL in the browser to test the API.



Expected Output:

Hello from Secure Serverless App!

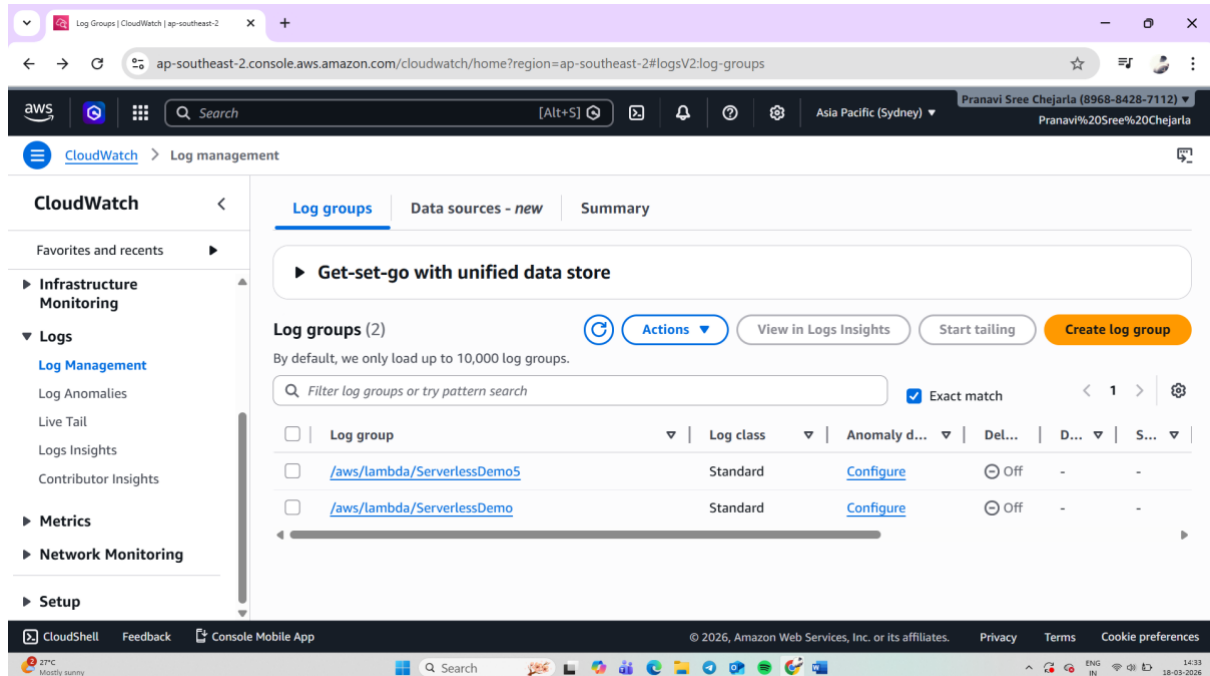
Hello from Secure Serverless App!

PART 6: Monitor Application Logs

To monitor API execution, AWS provides **CloudWatch Logs**.

Go to:

CloudWatch → Log Groups



Open the log group:

/aws/lambda/ServerlessDemo2

View execution logs.

Logs contain details such as:

START RequestId

END RequestId

REPORT Duration

The screenshot shows the AWS CloudWatch console interface. The top navigation bar includes the AWS logo, a search bar, and the user's profile information (Pranavi Sree Chejarla). The left sidebar contains the CloudWatch menu with options like Favorites and recents, Ingestion, Dashboards, Alarms, Application Signals (APM), Infrastructure Monitoring, and Logs. The main content area displays the log group details for `/aws/lambda/ServerlessDemo2`. The details are organized into three columns: Log class (Standard), ARN (`arn:aws:logs:ap-south-2:896884287112:log-group:/aws/lambda/ServerlessDemo2`), Creation time (41 minutes ago), Retention (Never expire), Stored bytes, Metric filters (0), Subscription filters (0), Contributor Insights rules, KMS key ID, Deletion protection (Off), Data protection, Sensitive data count, Custom field indexes, Transformer, and Anomaly detection. The bottom of the console shows the CloudShell, Feedback, and Console Mobile App links, along with the copyright notice for Amazon Web Services, Inc.

Practical Result

The serverless application was successfully implemented using **AWS Lambda and API Gateway**.

The system performs the following:

- Lambda function created and deployed
- API Gateway integrated with Lambda
- HTTP endpoint created and tested
- Basic API security configured
- Execution logs monitored using CloudWatch

✓ Outcome:

The screenshot shows the AWS CloudWatch console interface displaying the log events for the `/aws/lambda/ServerlessDemo2` log group. The top navigation bar includes the AWS logo, a search bar, and the user's profile information (Pranavi Sree Chejarla). The left sidebar contains the CloudWatch menu with options like Favorites and recents, Ingestion, Dashboards, Alarms, Application Signals (APM), Infrastructure Monitoring, and Logs. The main content area displays the log events for the selected log group. The events are listed in a table with columns for Timestamp and Message. The events show the start and end of a request, along with the duration and billed duration. The bottom of the console shows the CloudShell, Feedback, and Console Mobile App links, along with the copyright notice for Amazon Web Services, Inc.

Timestamp	Message
2026-03-02T08:29:44.982Z	START RequestId: fde46363-cd08-45f7-9b58-edfe76965fd2 Version: \$LATEST
2026-03-02T08:29:44.997Z	END RequestId: fde46363-cd08-45f7-9b58-edfe76965fd2
2026-03-02T08:29:44.997Z	REPORT RequestId: fde46363-cd08-45f7-9b58-edfe76965fd2 Duration: 1.86 ms Billed Duration: 2...
2026-03-02T08:30:51.124Z	START RequestId: 312e4fc2-a65f-4d12-9e85-492525c87463 Version: \$LATEST
2026-03-02T08:30:51.159Z	END RequestId: 312e4fc2-a65f-4d12-9e85-492525c87463
2026-03-02T08:30:51.159Z	REPORT RequestId: 312e4fc2-a65f-4d12-9e85-492525c87463 Duration: 34.68 ms Billed Duration: ...

